Stabilisation active de systèmes en équilibre instable: du pendule aux lanceurs réutilisables

Thibaut Dehaeze (26351)

Épreuve de TIPE

Session 2026

Introduction

Applications

Lanceur Réutilisable [1]

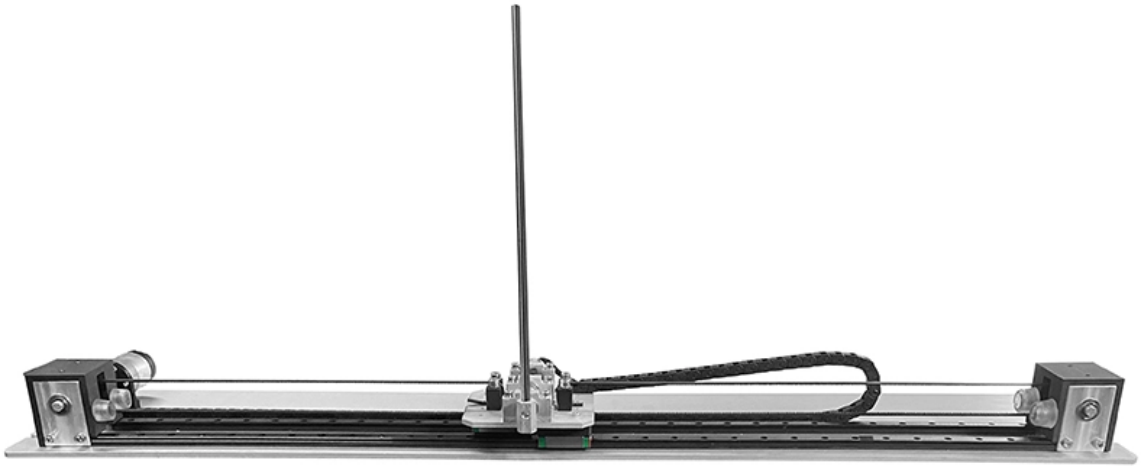
Segway [2]

Robot auto-équilibré [3]

Introduction

Le Pendule Inversé

Pendule inversé rotatif [4]



Pendule inversé sur chariot [5]

Introduction

Problématique

Comment la dynamique d'un système mécanique au voisinage d'un équilibre instable peut-elle être modifiée afin d'assurer sa stabilité au cours du temps, comme dans le cas des lanceurs réutilisables lors de leur retour vertical ?

Plan de l'exposé

Démarche

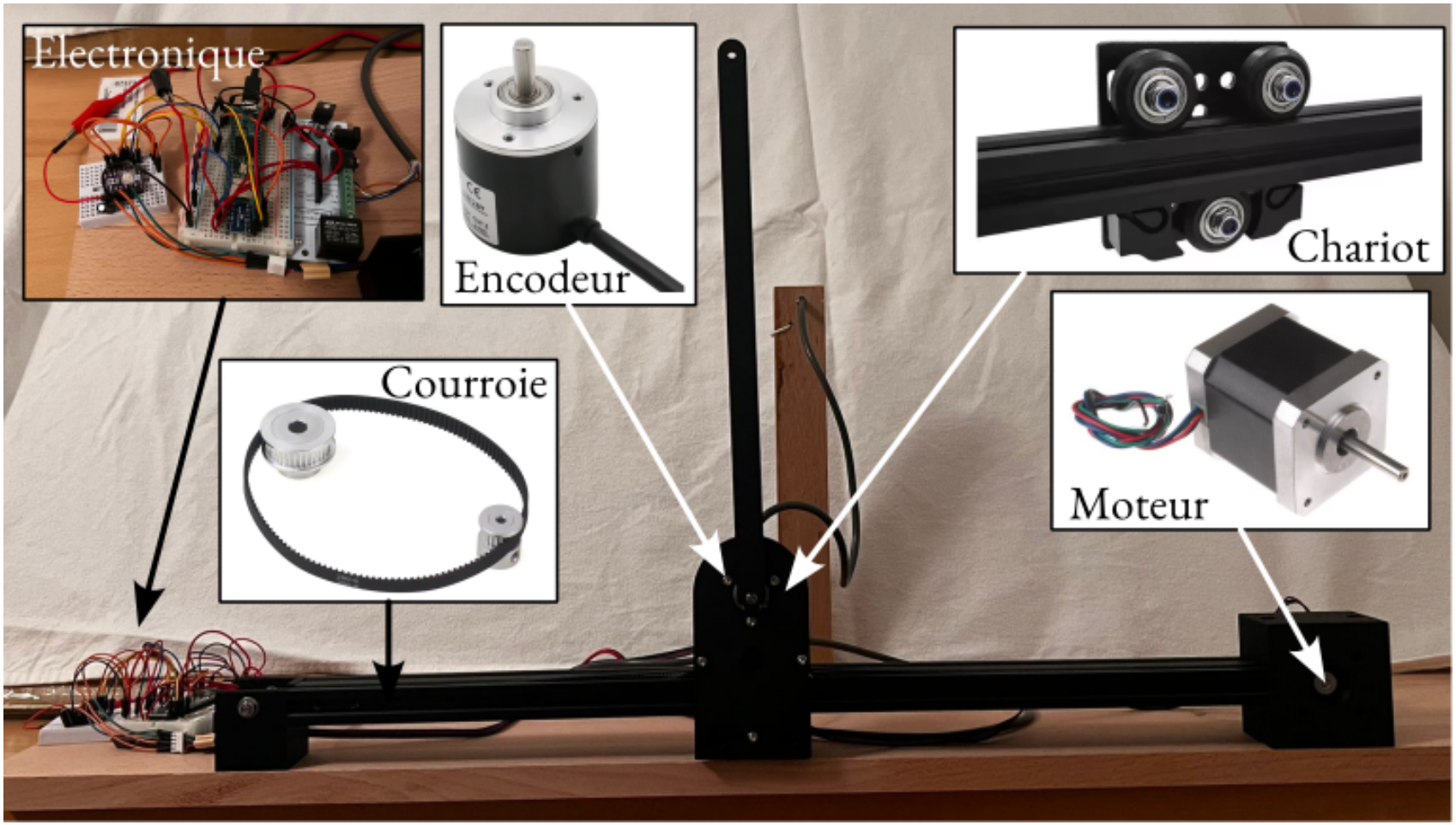
1. Introduction
2. Étude de la dynamique du Pendule Inversé
3. Stabilisation de la dynamique par contrôle PD
4. Stabilisation de la dynamique par contrôle LQ
5. Conclusion

Étude de la dynamique

Présentation du système

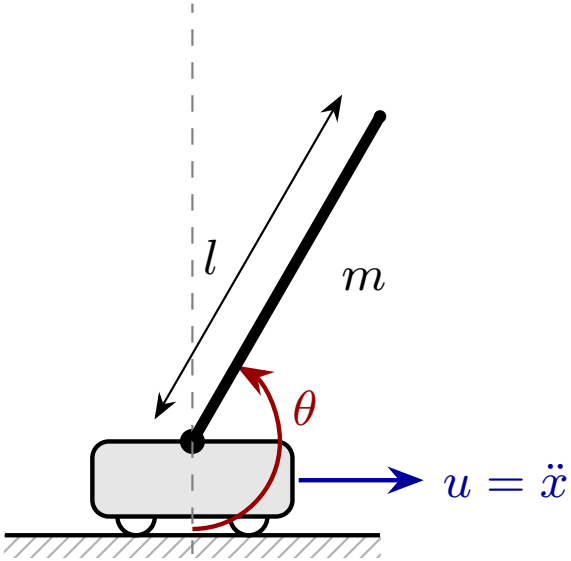
Étude de la dynamique

Présentation du système



Étude de la dynamique

Modèle et Équations de la dynamique

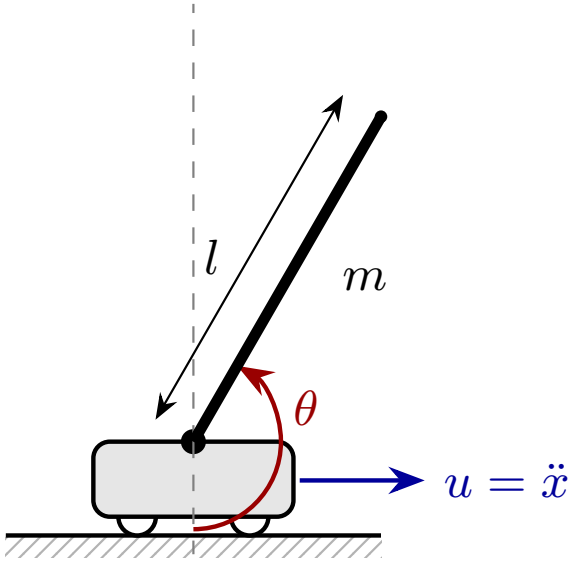


Équation du mouvement :

$$\frac{1}{3} m l^2 \, \ddot{\theta} + c \, \dot{\theta} + \underbrace{m g l \sin\theta + m l \cos\theta \cdot u}_{\text{non linéaire}} = 0$$

Étude de la dynamique

Modèle et Équations de la dynamique



Équation du mouvement :

$$\frac{1}{3} m l^2 \, \ddot{\theta} + c \, \dot{\theta} + \underbrace{m g l \sin\theta + m l \cos\theta \cdot u}_{\text{non linéaire}} = 0$$

Linéarisation pour étude de la dynamique et contrôle

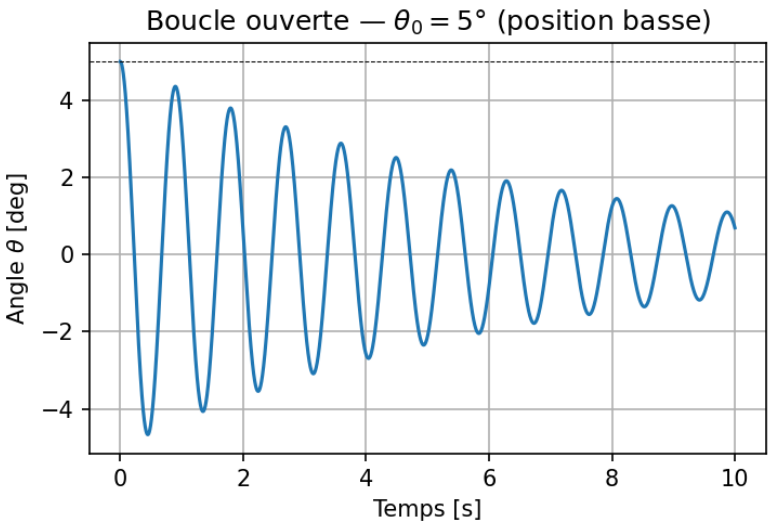
Étude de la dynamique

Linéarisations et Réponses du système

Position basse $(\theta \approx 0)$

$$\frac{1}{3} m l^2 \, \ddot{\theta} + c \, \dot{\theta} + m g l \, \theta = -m l u$$

$\Rightarrow$ Oscillations amorties



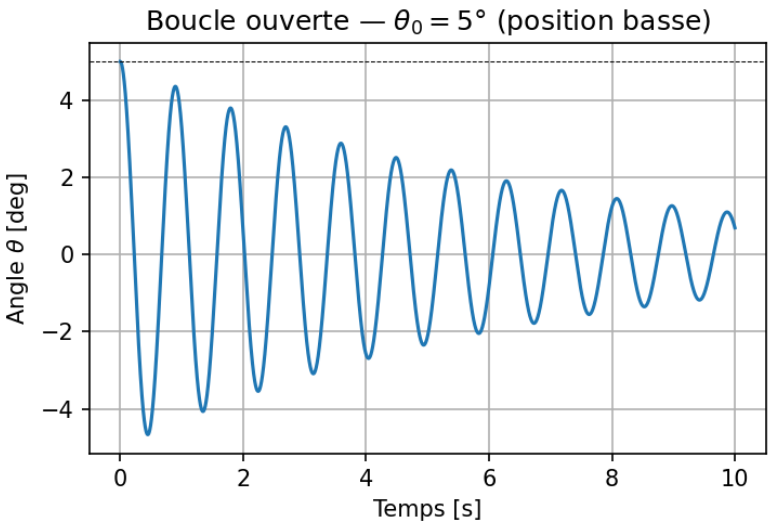
Étude de la dynamique

Linéarisations et Réponses du système

Position basse $(\theta \approx 0)$

$$\frac{1}{3} m l^2 \ddot{\theta} + c\dot{\theta} + mgl\,\theta = -mlu$$

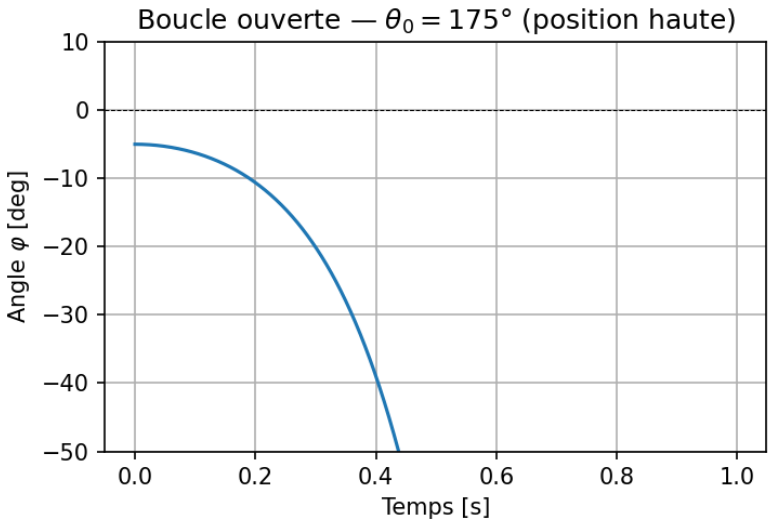
$\Rightarrow$ Oscillations amorties



Position haute $(\varphi = \theta - \pi \approx 0)$

$$\frac{1}{3} m l^2 \ddot{\varphi} + c\dot{\varphi} - mgl\,\varphi = mlu$$

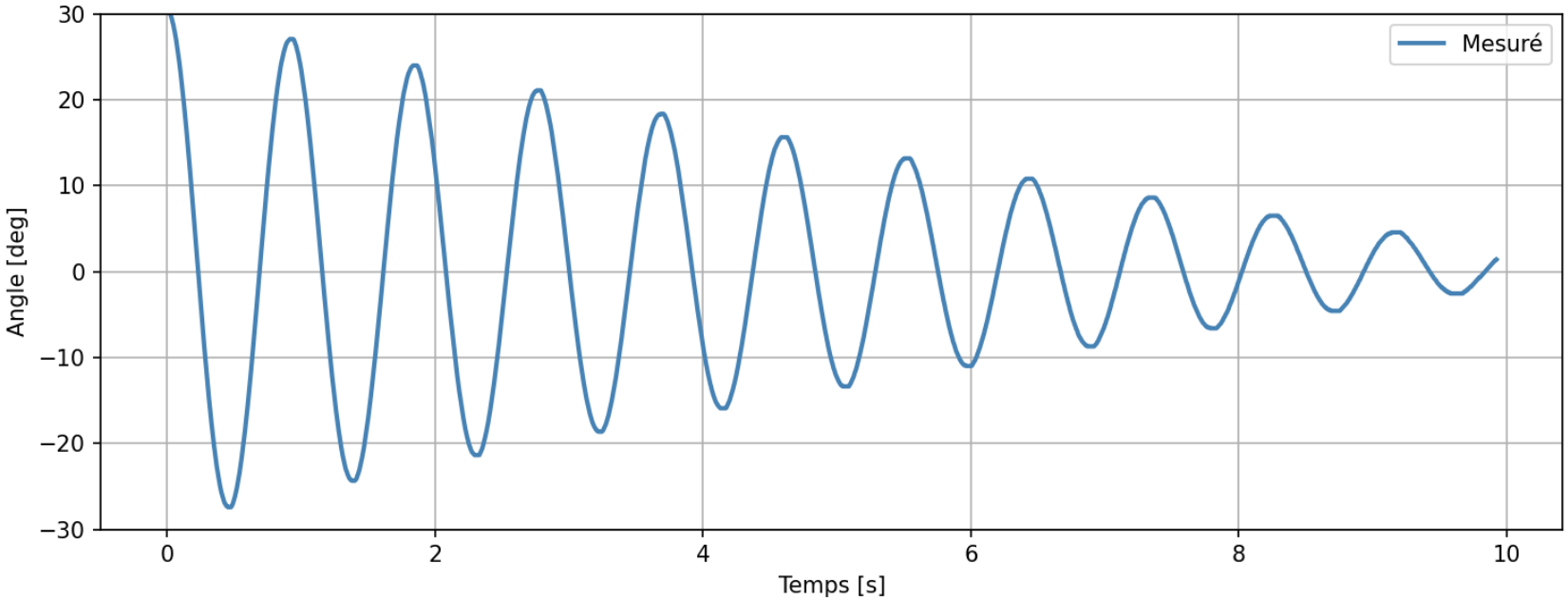
$\Rightarrow$ Divergence exponentielle



Étude de la dynamique

Comparaison modèle / expérience

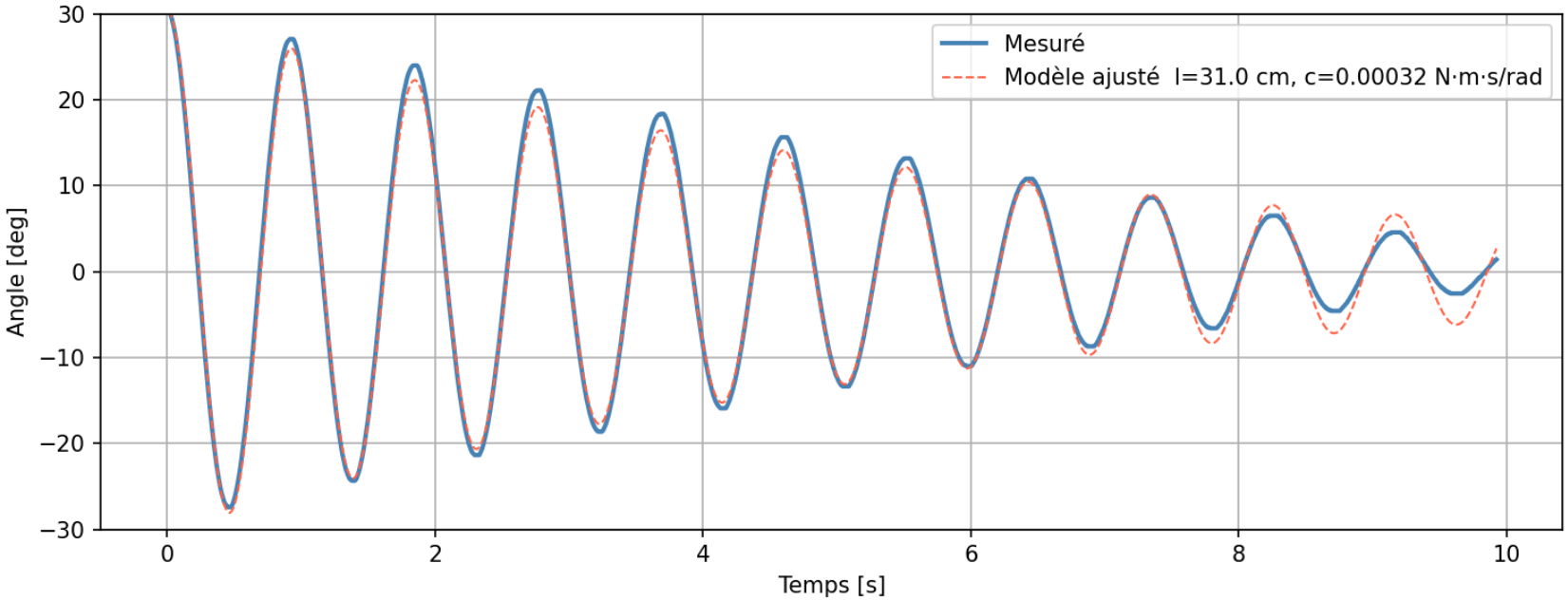
Mesure de la réponse expérimentale



Étude de la dynamique

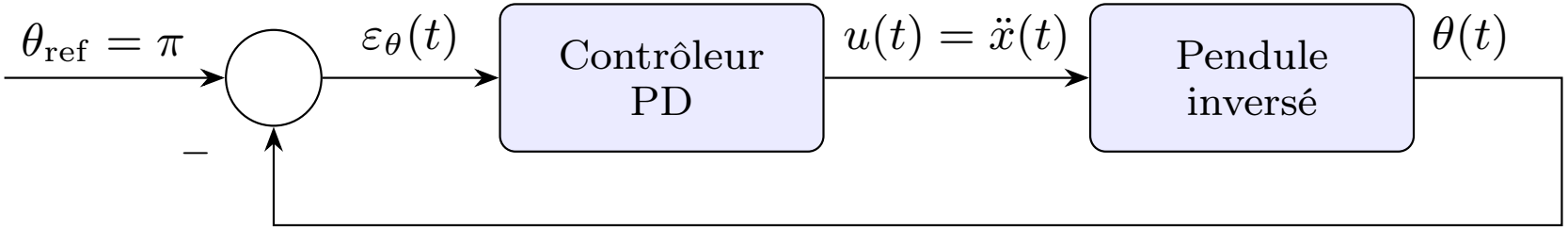
Comparaison modèle / expérience

Mesure de la réponse expérimentale $\xrightarrow[m = 30\,g]{\text{Données}}$ Optimisation des paramètres du modèle (Python) $\longrightarrow$ Paramètres identifiés $l,\ c$



Contrôle PD

Schéma block du contrôle du pendule inversé



$$u(t) = \underbrace{k_p \cdot \varepsilon_\theta(t)}_{\text{Proportionnel}} + \underbrace{k_d \cdot \frac{\mathrm{d}\varepsilon_\theta(t)}{\mathrm{d}t}}_{\text{Dérivé}}$$

Contrôle PD

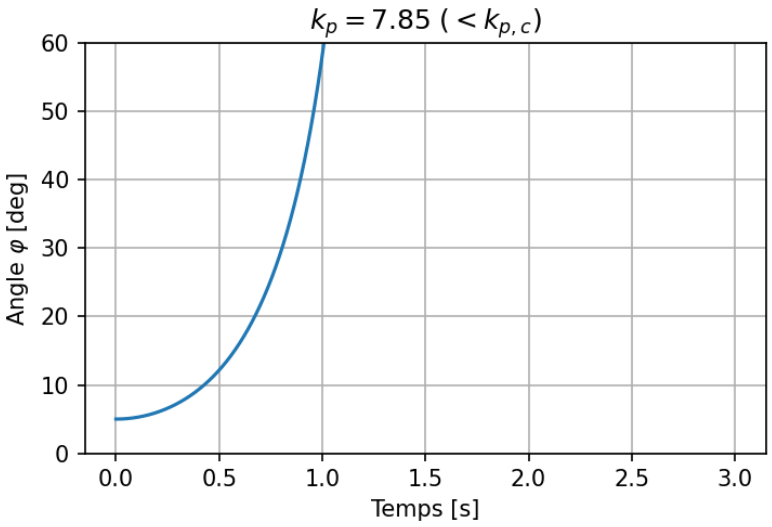
Contrôleur Proportionnel

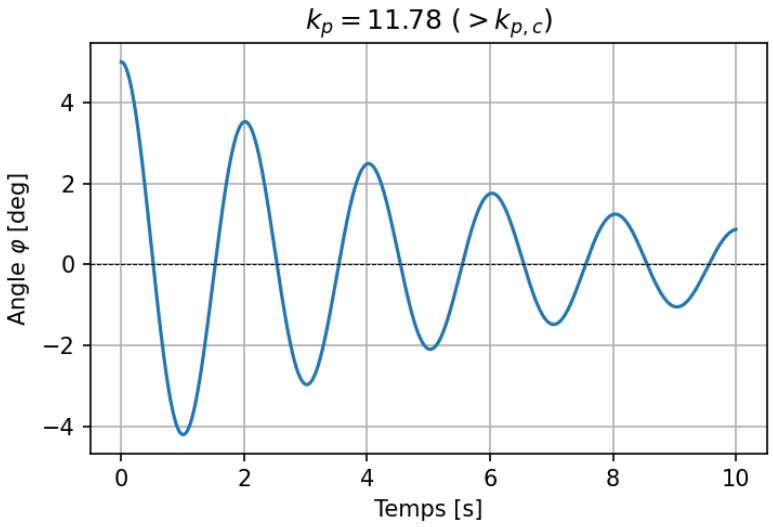
Avec $u = -k_p\,\varphi$, l'équation linéarisée devient :

$$\frac{1}{3}ml^2\,\ddot{\varphi} + c\,\dot{\varphi} + (ml\,k_p - mgl)\,\varphi = 0$$

Stable si $\Delta = c^2 - 4\frac{1}{3}ml^2 \cdot (ml\,k_p - mgl) > 0$, donc si $k_p > k_{p,c}$, avec

$$k_{p,c} = g + \frac{3c^2}{4m^2l^3} \approx 9.8$$





Contrôle PD

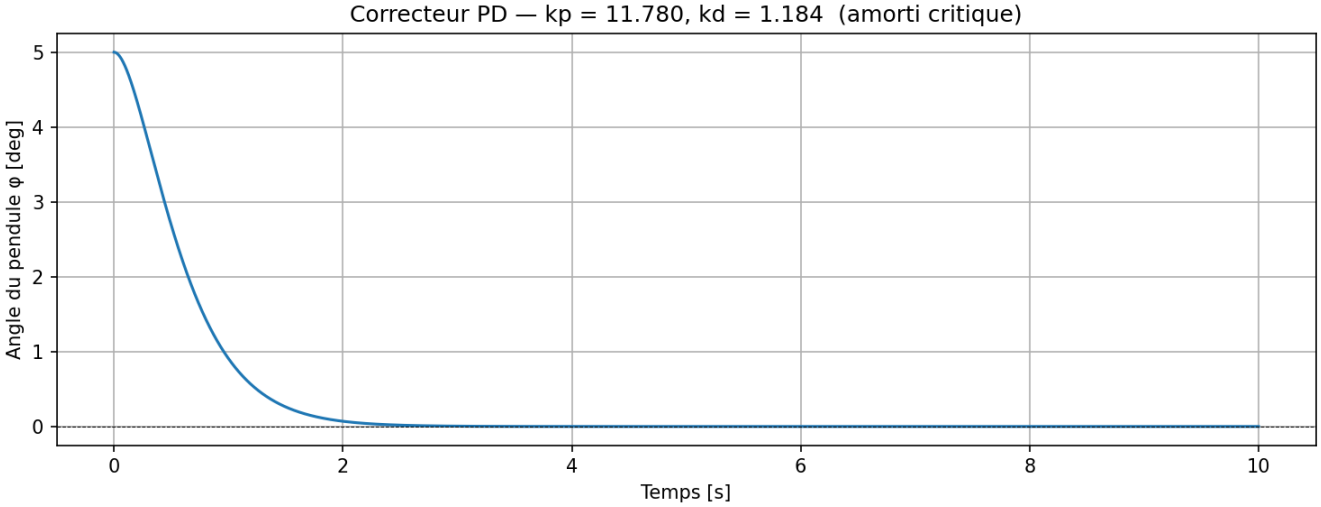
Contrôleur Dérivatif

Avec $u = -k_p\,\varphi - k_d\,\dot{\varphi}$, l'équation linéarisée devient :

$$\frac{1}{3}ml^2\,\ddot{\varphi} + (c + ml\,k_d)\,\dot{\varphi} + (ml\,k_p - mgl)\,\varphi = 0$$

Amortissement critique :

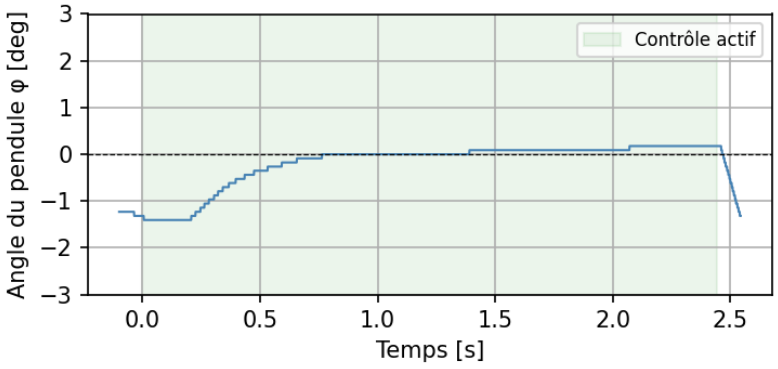
$$k_{d,c} = \frac{1}{ml}\left(2\sqrt{\frac{1}{3}ml^2\,(ml\,k_p - mgl)} - c\right) \approx 1.2$$



Contrôle PD

Résultats expérimentaux

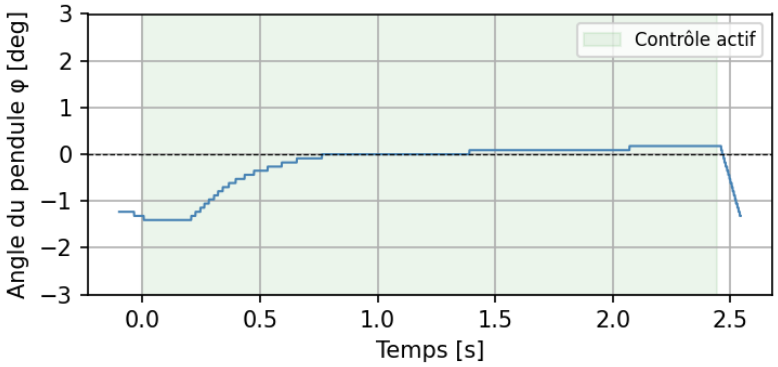
Résultats expérimentaux — $k_p = 12$, $k_d = 1$

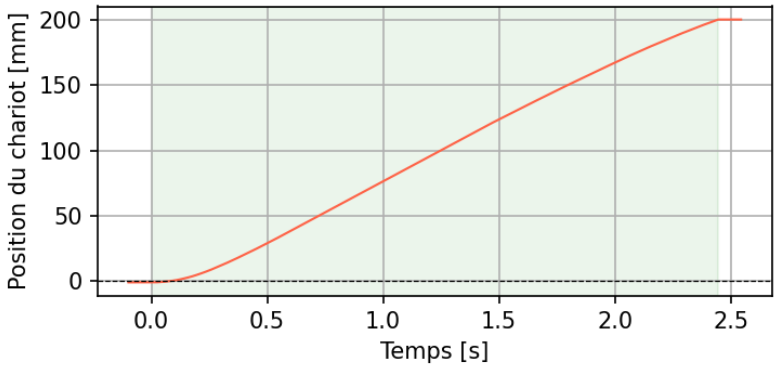


Contrôle PD

Résultats expérimentaux

Résultats expérimentaux — $k_p = 12$, $k_d = 1$

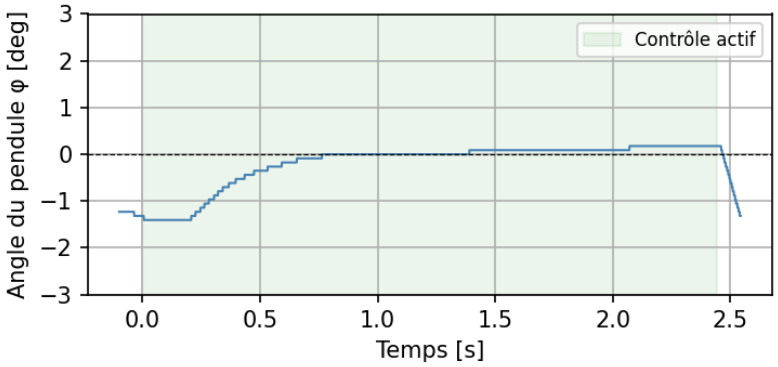


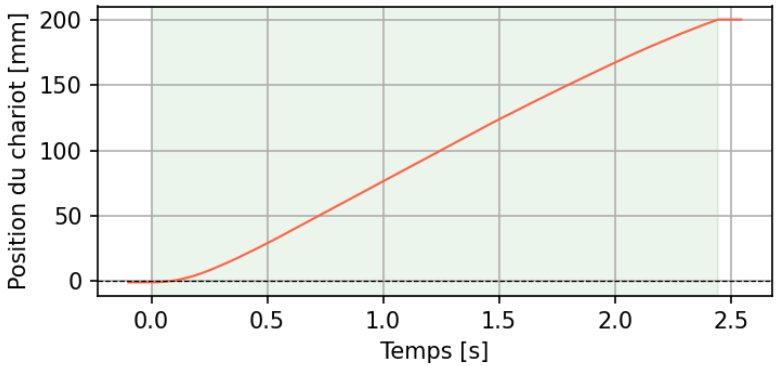


Contrôle PD

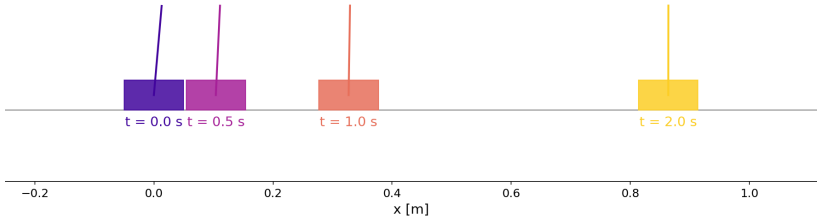
Résultats expérimentaux

Résultats expérimentaux — $k_p = 12$, $k_d = 1$





Même comportement en simulation : ϕ est correctement stabilisé, mais le chariot n'est pas stable.



Résultats de simulation — $k_p = 12$, $k_d = 1$

Pour contrôler à la fois ϕ et x, le contrôleur PD n'est pas adapté.

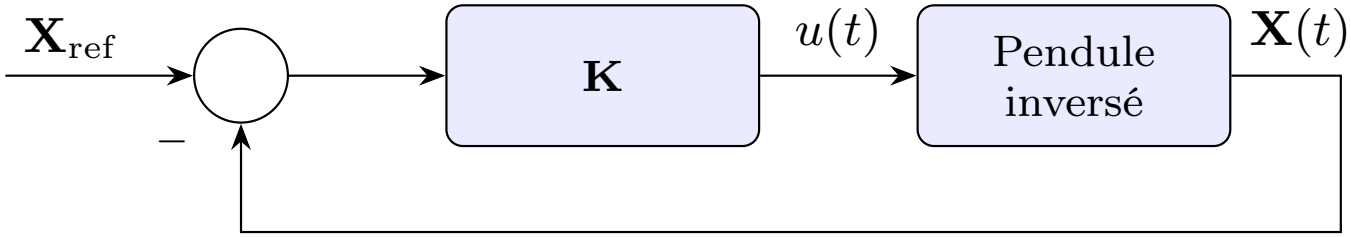
Contrôle LQ

Modèle d'état — Commande par retour d'état

Modèle par représentation d'états :

$$\mathbf{X} = \begin{pmatrix} x \\ \dot{x} \\ \varphi \\ \dot{\varphi} \end{pmatrix}, \quad \dot{\mathbf{X}} = A\mathbf{X} + Bu, \quad A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{mgl}{D} & -\frac{c}{D} \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 1 \\ 0 \\ \frac{ml}{D} \end{pmatrix}$$

Commande par retour d'état :



$$u(t) = \mathbf{K}\left(\mathbf{X}_{\text{ref}} - \mathbf{X}(t)\right), \quad \mathbf{K} = \begin{bmatrix} k_x & k_{\dot{x}} & k_\varphi & k_{\dot{\varphi}} \end{bmatrix}$$

Contrôle LQ

Choix de la matrice de retour d'état

Minimisation d'un critère quadratique :

$$J = \int_0^\infty \left(\mathbf{X}^\top Q\,\mathbf{X} + u^\top R\,u\right) \mathrm{d}t$$

- $Q \succeq 0$ pondère les états : $Q = \mathrm{diag}(q_x,\, q_{\dot{x}},\, q_\varphi,\, q_{\dot{\varphi}})$
- $R > 0$ pondère l'effort de commande

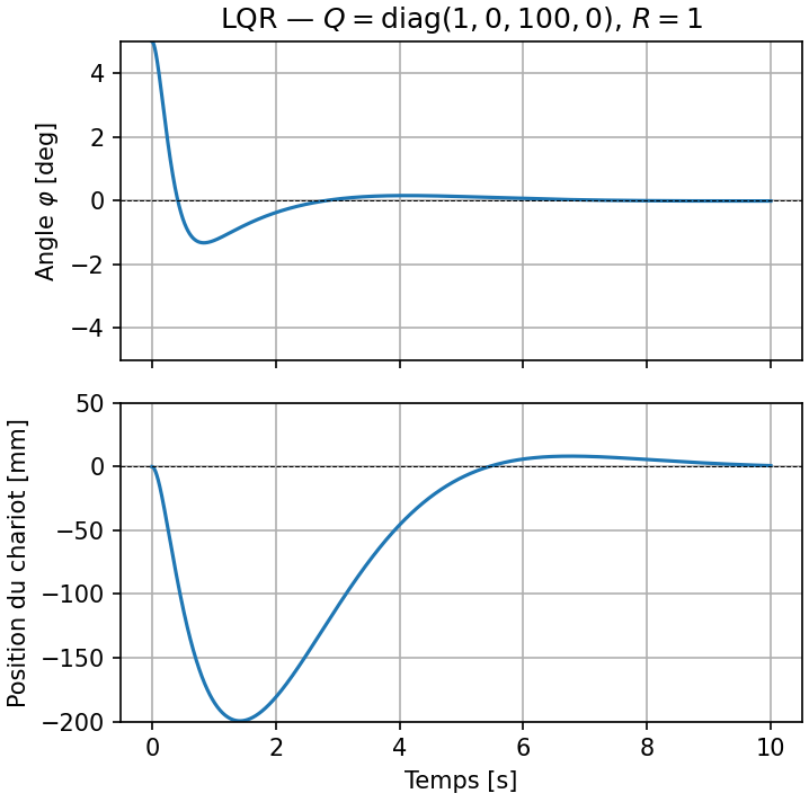
$\mathbf{K}$ est obtenu par résolution de l'équation algébrique de Riccati :

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0 \implies \boxed{\mathbf{K} = R^{-1}B^\top P}$$

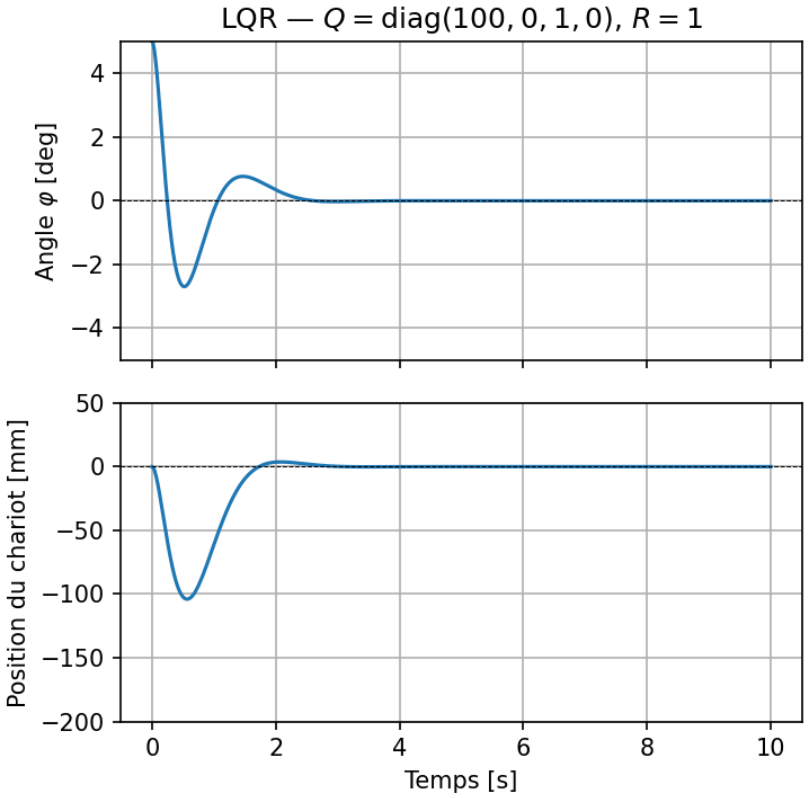
Exemple $q_x = 100,\, q_\varphi = 1,\, R = 1 \implies \mathbf{K} = \begin{bmatrix} -10.0 & -8.7 & 46.9 & 9.8 \end{bmatrix}$

Contrôle LQ

Simulation



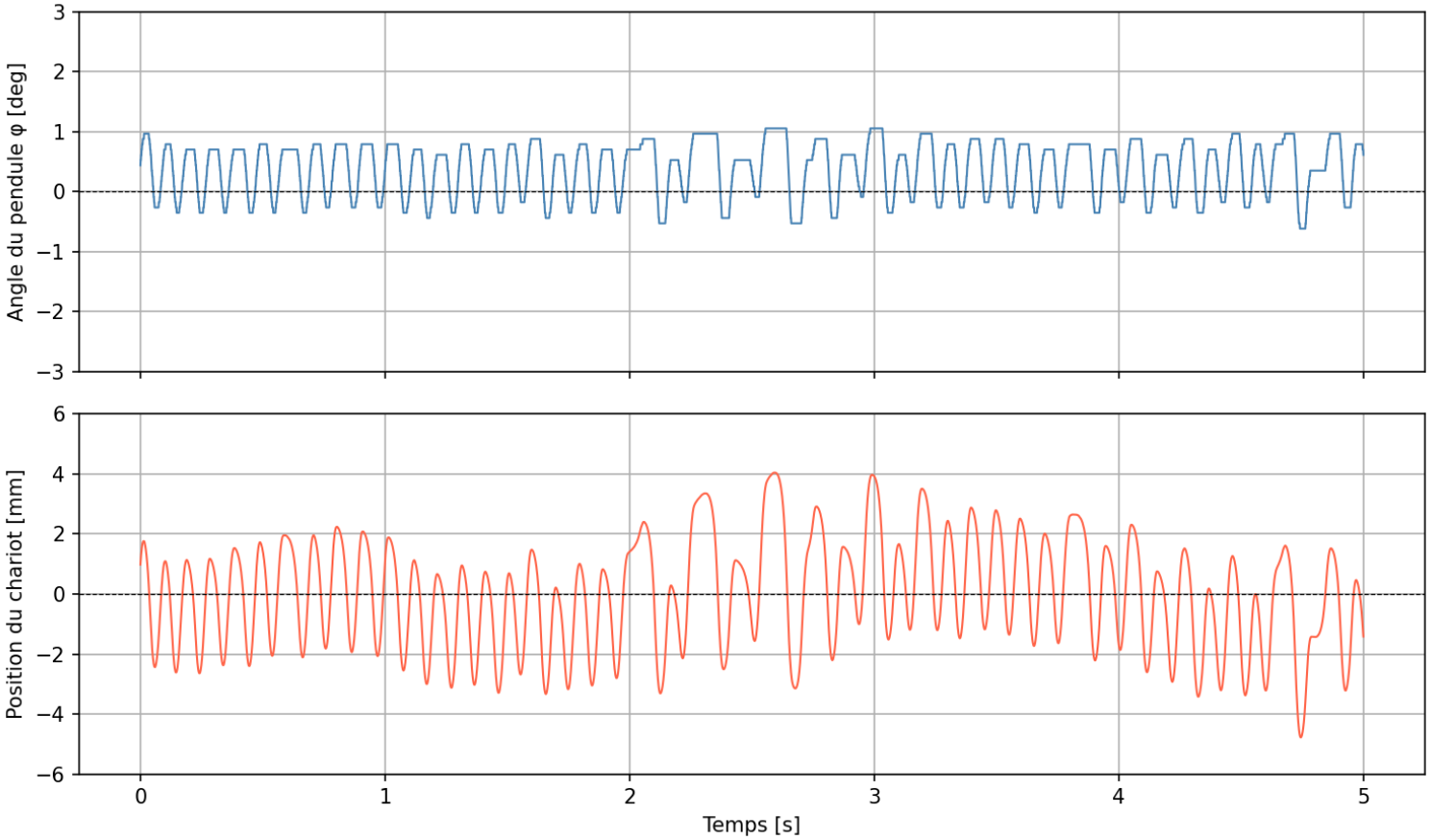
Pondération forte sur φ : $q_x = 1$, $q_\varphi = 100$



Pondération forte sur x : $q_x = 100$, $q_\varphi = 1$

Contrôle LQ

Résultats Expérimentaux



$\mathbf{K} = [-31.6 \;\; -23.1 \;\; 92.8 \;\; 18.3]$, stabilité observée de $\pm 1\,\text{deg}$, $\pm 4\,mm$

Conclusion

Bilan

- Pendule inversé sur chariot comme support d'étude du contrôle actif d'une dynamique instable
- Deux stratégies de contrôle comparées :
  - **PD** — intuitif, paramètres à sens physique clair (gain, amortissement), mais ne stabilise pas la position du chariot
  - **LQR** — optimal au sens d'un critère quadratique, s'appuie sur un modèle linéarisé, contrôle simultané de φ et x

Perspectives

- **LQG** — ajout d'un filtre de Kalman pour l'estimation des états bruités ($\dot{x}$, $\dot{\varphi}$) ; extension naturelle du LQR
- **MPC** (commande prédictive) — optimisation sur un horizon glissant, permet de prendre en compte explicitement certaines contraintes physiques

Références I

[1] *Programme de développement de lanceurs réutilisables – SpaceX*. URL : https://fr.wikipedia.org/wiki/Programme_de_d%C3%A9veloppement_de_lanceurs_r%C3%A9utilisables_de_SpaceX (visité le 04/03/2026).

[2] *Gyropode*. URL : https://fr.wikipedia.org/wiki/Gyropode (visité le 04/03/2026).

[3] *Boston Dynamics – History*. URL : https://bostondynamics.com/about/history/ (visité le 04/03/2026).

[4] *Quanser – Inverted Rotary Pendulum*. URL : https://edu4industry.com/en/produkt/quanser-inverted-rotary-pendulum/ (visité le 04/03/2026).

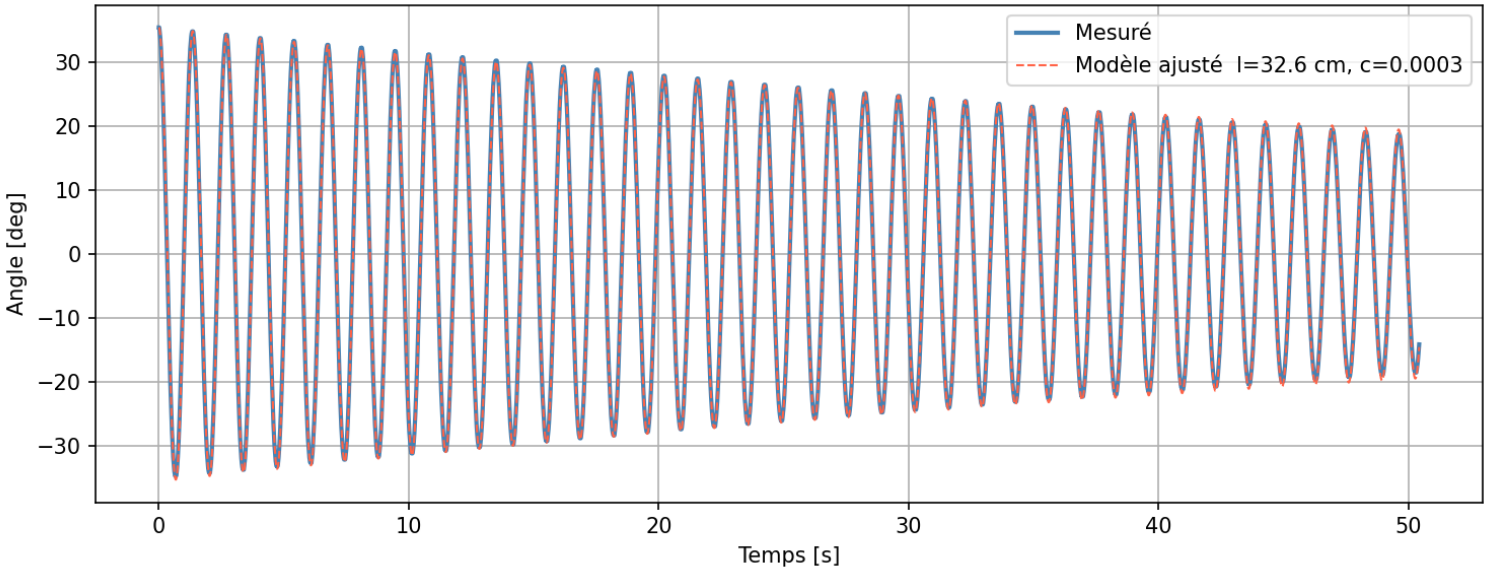
[5] *Acrome – Linear Inverted Pendulum*. URL : https://acrome.net/product/linear-inverted-pendulum (visité le 04/03/2026).

Annexe — Matériel

Récupération d'une imprimante 3D Alfawise U20

Annexe — Identification du modèle

Réponse libre — masse 100 g



Avec une masse plus importante, la friction est négligeable devant l'inertie : le modèle linéarisé reproduit fidèlement la réponse libre expérimentale.

Annexe — Code

Identification des paramètres du modèle (tools/fit.py)

Minimisation des moindres carrés entre simulation et mesure :

```
def simulate_free(t_eval, theta0, omega0, l, c, m):
    D = (1/12)*m*(2*l)**2 + m*l**2    # = 4/3 * m * l^2
    def ode(t, y):
        theta, omega = y
        return [omega, (-c*omega - m*G*l*np.sin(theta)) / D]
    sol = solve_ivp(ode, (t_eval[0], t_eval[-1]), [theta0, omega0], t_eval=t_eval)
    return sol.y[0]

def cost(params, t, theta_meas, m):
    l, c = params
    theta_sim = simulate_free(t, theta_meas[0], 0.0, l, c, m)
    return float(np.mean((theta_sim - theta_meas)**2))

result = minimize(cost, [l_init, c_init], args=(t, theta, m), method="Nelder-Mead")
l_fit, c_fit = result.x
```

Algorithme de Nelder-Mead sur les paramètres (l, c), masse m fixée.

Annexe — Code

Simulation du système (model/inverted_pendulum.py)

Dynamique non-linéaire :

```
def dynamics(self, state, u):
    x, v, theta, omega = state
    D = p.I + p.m*p.l**2
    domega = (- p.c*omega
          - p.m*p.g*p.l*np.sin(theta)
          - p.m*p.l*u*np.cos(theta)) / D
    return np.array([v, u, omega, domega])
```

Intégration Euler, $dt = 1\,\text{ms}$:

```
for k in range(N):
    u = controller(t[k], x)
    dx = plant.dynamics(x, u)
    x  = x + dx * dt
    history[k] = x
```

Synthèse LQR :

```
Q = np.diag([q_x, 0, q_phi, 0])
R = np.array([[1]])
K, P, E = control.lqr(A, B, Q, R)
```

Contrôleur LQR :

```
def __call__(self, t, state):
    phi_state = state.copy()
    phi_state[2] = state[2] - np.pi
    u = float((-self.K @ phi_state).item())
    return u
```

Annexe — Code

Contrôleur embarqué — Teensy 4.1 (teensy/src/main.cpp)

Boucle de contrôle à 1 kHz via `IntervalTimer` :

```
// Convert to SI units
float phi_rad     = (pend_deg   - theta_ref_deg) * (PI / 180.0f);
float phi_dot_rad =  pend_vel_deg_s              * (PI / 180.0f);
float x_m         = (cart_mm    - x_ref_mm)      / 1000.0f;
float x_dot_m     =  cart_vel_mm_s               / 1000.0f;

// Controller output [m/s^2]
switch (ctrl_mode) {
    case MODE_PD:
        accel_m_s2 = -Kp * phi_rad - Kd * phi_dot_rad;  break;
    case MODE_LQR:
        accel_m_s2 = -(K[0]*x_m + K[1]*x_dot_m + K[2]*phi_rad + K[3]*phi_dot_rad);  break;
}

// Soft braking: block acceleration toward cart limits (+/-160 mm)
if (cart_mm >  CART_SOFT_MM && accel_cmd > 0.0f) accel_cmd = 0.0f;
if (cart_mm < -CART_SOFT_MM && accel_cmd < 0.0f) accel_cmd = 0.0f;

cart_vel_mm_s += accel_cmd * DT;   // integrate acceleration -> velocity
```